

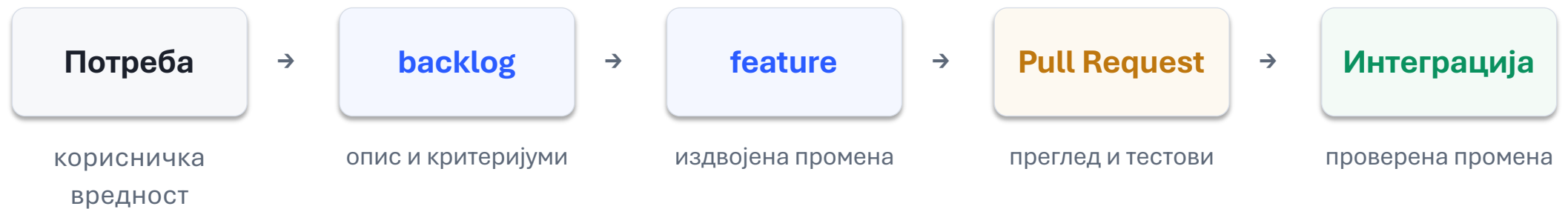


Захтеви, backlog, Git и тимски развој

Пословна потреба постаје проверљива промена у систему

Развојни ток је једна повезана целина

Захтев, планирање рада и Git не треба посматрати као одвојене теме.



КЉУЧНА ПОРУКА

Циљ је да тим повеже захтев, одлуку, имплементацију и доказ да све ради исправно.

User Story описује вредност

Добар запис одговара на питања ко, шта и зашто.

**Као наставник лабораторије,
желим да резервишем слободну опрему за одређени термин,
како бих знао да је опрема доступна и спречио двоструку
резервацију.**

КЉУЧНА ПОРУКА

Форма има смисла само ако иза ње стоји проверљиво понашање система.

Критеријуми прихватања претварају захтев у доказ

Сваки критеријум треба да буде проверљив тестом или демонстрационим сценаријем.

Тип сценарија	Пример критеријума
Успешан ток	Резервација се креира када је опрема доступна и период исправан.
Пословни конфликт	Систем одбија период који се преклапа са активном резервацијом.
Неисправан улаз	Почетак термина мора бити пре краја.
Гранични случај	Правило јасно дефинише термин краја и почетка термина.

Backlog није архива идеја

Ставка спремна за рад има циљ, приоритет, критеријуме и позната ограничења.

01

Product Backlog

све будуће ставке

02

Sprint Backlog

изабрани обим рада

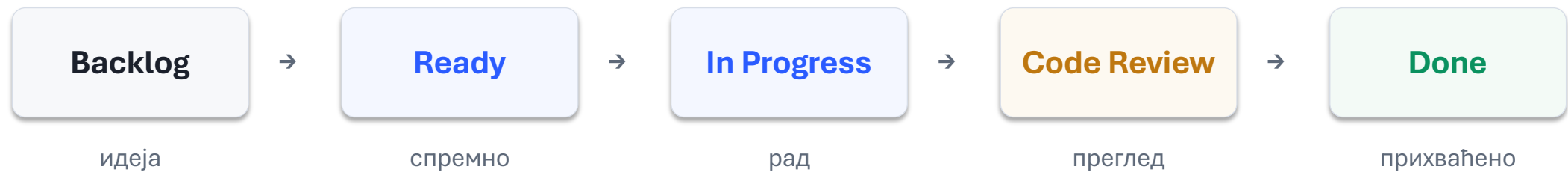
03

Increment

реализована промена

Договорени ток рада мора бити видљив

Статус картице мора да одговара стварном стању посла.



КЉУЧНА ПОРУКА

Done значи да су испуњени критеријуми, извршене провере и интегрисана одговарајућа промена.

Једна backlog ставка има више од наслова

Контекст, договор и траг одлука треба да буду на једном месту.

01

Опис

шта се тражи и ко добија вредност

02

Критеријуми

како се доказује исправност

03

Приоритет и sprint

зашто се ради сада

04

Assignee и коментари

ко ради и шта је договорено

Велики задатак треба разбити на мале проверљиве кораке

Checklist није бирократија. Она смањује ризик и чини напредак видљивим.

ПРИМЕР КОРАКА

wireframe или ток екрана

имплементација основне логике

тестови кључног правила

code review и дорада

ЗАШТО ЈЕ ТО ВАЖНО

мање ризика

видљив застој

лакша провера

лакше образложење

Git је запис намере, не резервна копија

Развој треба моћи реконструисати из малих и смислених промена.

Појам	Улога у раду тима
Working tree	датотеке које се тренутно мењају
Staging area	избор садржаја за наредни commit
Commit	кохерентна промена са описом намере
Branch	издвојена линија развоја
Remote	удаљени репозиторијум за размену промена

Мали развојни корак изгледа овако

Команде су мање важне од дисциплине рада око њих.

```
1 git status
2 git diff
3 git add src/Application/ReservationService.cs
4 git add tests/ReservationServiceTests.cs
5 git diff --staged
6 git commit -m "Add reservation overlap rule"
7 git push -u origin feature/reservation-overlap
```

Шта је важно

Пре commit-а прегледати припремљени diff.

Један commit треба да има једну кохерентну намеру.

Неповезане промене не иду у исти Pull Request.

Главна грана остаје стабилна за цео тим.

Pull Request повезује захтев са имплементацијом

Преглед се не ради после рада. Он је део рада.

01

Обим

да ли промена решава тачну ставку

02

Архитектура

да ли се поштују договорене границе

03

Тестови

да ли постоји доказ провере

04

Ризици

шта се мења и шта може поћи по злу

Конфликт при спајању је одлука о понашању

Решење конфликта није само уклањање техничких маркера.

```
1 git checkout main
2 git pull
3 git checkout feature/reservation-overlap
4 git merge main
5 # решити конфликт и прегледати diff
6 git add src/Application/ReservationService.cs
7 git commit -m "Resolve reservation rule conflict"
8 dotnet test
```

Шта је важно

Одлучити које понашање мора да остане.

Последњи корак је провера да систем и даље ради.

Force push није уобичајен ток на дељеним гранама.

P1 је први доказ да тим ради контролисано

Не очекује се велика имплементација. Очекује се следљив процес.

01

Репозиторијум и README

заједнички репозиторијум и основни опис

02

Почетни backlog

јасне ставке и приоритети

03

Грана или PR

ставка повезана са конкретним радом

04

Докази провере

тим уме да објасни шта је урађено

Одговорност за рад остаје код студента

AI алати су дозвољени, али нису доказ разумевања.

ПОТРЕБНО ЈЕ РАЗГРАНИЧИТИ

сопствени рад

рад осталих чланова тима

садржај добијен спољним алатима

РЕЛЕВАНТАН ДОКАЗ РАДА

репозиторијум и историја измена

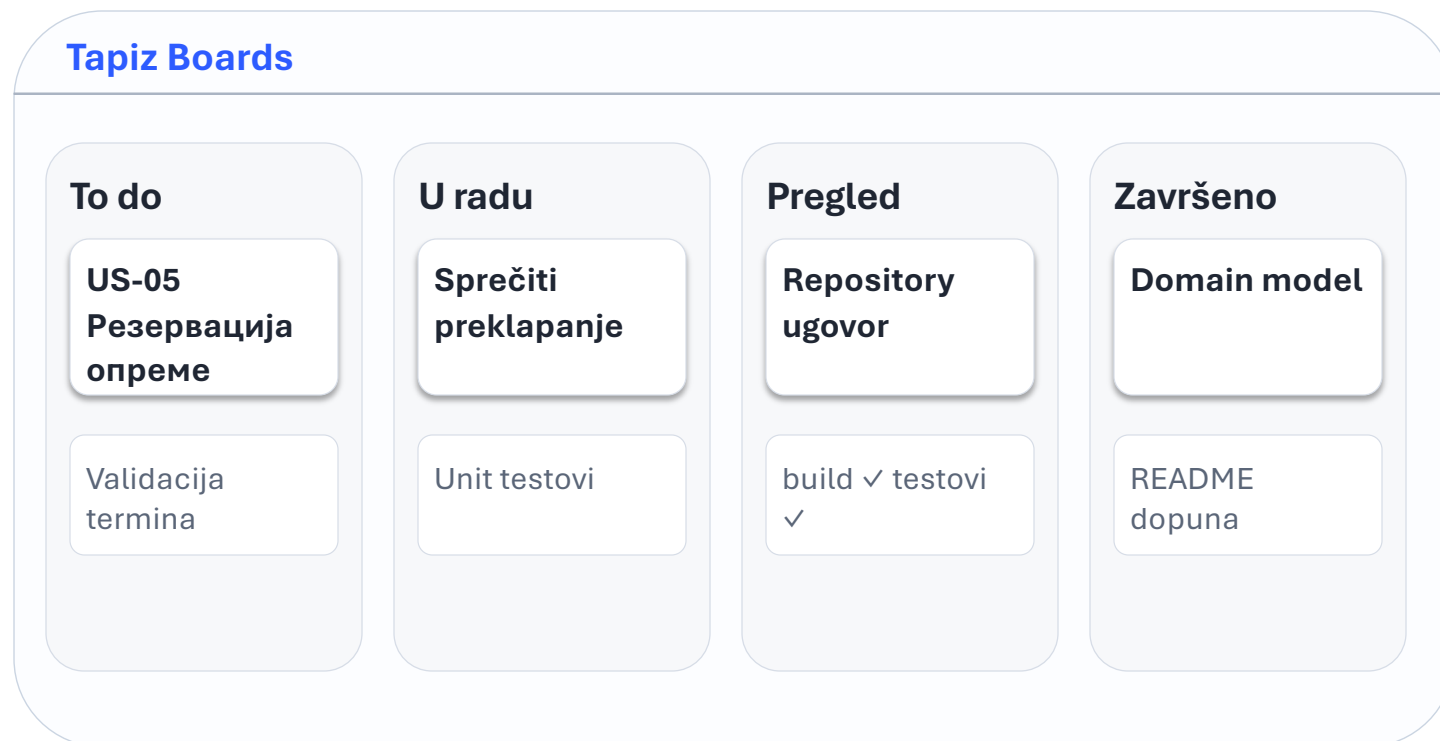
тестови и документација

евиденција употребе AI алата

самостално образложење

Tapiz Boards је радни договор тима

Табла није само списак картица, већ видљив ток од идеје до прихваћене промене.



Како се чита табла

Колоне показују стварно стање рада, не жељу тима.

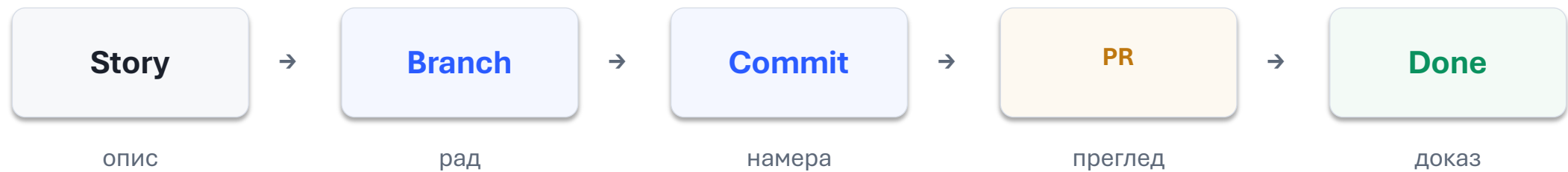
Картица треба да има опис, приоритет, assignee и критеријуме прихватања.

Веза са PR-ом и тестовима показује да је промена проверљива.

Done значи прихваћено по Definition of Done, не само померено у последњу колону.

Од backlog ставке до интегрисане промене

Сваки корак оставља траг који тим може да покаже.



КЉУЧНА ПОРУКА

Нема професионалног развојног тока без повезивања алата, договора и провере.

Провера пре предаје рада

Пре него што се рад прогласи завршеним, тим треба да провери основне сигнале.

- 01 Да ли је ставка повезана са граном или Pull Request-ом?
- 02 Да ли критеријуми прихватања имају видљиву проверу?
- 03 Да ли diff садржи само релевантне измене?
- 04 Да ли је главна грана стабилна?
- 05 Да ли сваки члан може да објасни свој допринос?

Суштина Вежбе 1

Први практични циљ је дисциплина рада, а не количина написаног кода.

Професионалан тимски рад почиње када се захтев може пратити до кода, теста, review-а и интеграције.

КЉУЧНА ПОРУКА

То је основа за све наредне вежбе.

Најчешћи пропусти у првом току рада

Ови пропусти касније постају скупии јер прекидају следљивост рада.

01

Нејасан Story

није јасно ко добија вредност

02

Без критеријума

не зна се када је завршено

03

Велики PR

review губи фокус

04

Без доказа

нема теста, diff-а или објашњења

За наредни термин

Резултат вежбе треба да буде употребљив у пројекту.

- 01 Написати User Story за једну ставку пројектног домена.
- 02 Додати успешне, негативне и граничне критеријуме прихватања.
- 03 Увести ставку у Tapiz ток и радити преко feature гране.
- 04 Отворити Pull Request са доказом провере.
- 05 Објаснити цео пут од захтева до интегрисане промене.